PRACTICAL NO : 1

Name : Ghanwat Payal Ganesh Roll No : 222 Class : SE (AIML) Batch:S1

Problem Statement: Searching and Sorting -- CO1, CO2

Design a program to maintain a student database that performs the following tasks:

1. Add and store student details (ID, Name, CGPA) using dynamically allocated memory.
2. Expand the student list using realloc() as new entries are added.
3. Implement Linear Search and Binary Search to find student records by ID.
4. Implement at least two Sorting Algorithms (Bubble Sort, Selection Sort, or Insertion Sort) to sort student records by:
    a. Name (Alphabetically)
    b. CGPA (Ascending/Descending)
5. Analyze and compare the performance of search operations before and after sorting.

Code :

```
#include <iostream>
#include <cstdlib>
#include <cstring>
#include <chrono>
using namespace std;
using namespace std::chrono;
struct Student
{
int id;
char name[50];
float cgpa;
}; int main()
{
Student *students = NULL;
```

```
int n;
cout << "Enter Number of Students: ";
cin >> n;
// Dynamic memory allocation using realloc()
for(int i = 0; i < n; i++)
{
students = (Student*)realloc(students, (i + 1) * sizeof(Student));
cout << "\nStudent " << i + 1 << endl;
cout << "Enter ID: ";
cin >> students[i].id;
cout << "Enter Name: ";
cin >> students[i].name;
cout << "Enter CGPA: ";
cin >> students[i].cgpa;
}
// Display Records
cout << "\n========== Student Records ==========\n";
cout << "ID\tName\tCGPA\n"; for(int i = 0; i < n; i++)
{
cout << students[i].id << "\t"
<< students[i].name << "\t"
<< students[i].cgpa << endl;
}
// Linear Search
int key;
cout << "\nEnter Student ID to Search (Linear Search): ";
cin >> key;
auto start = high_resolution_clock::now();
```

```
int pos = -1;
for(int i = 0; i < n; i++)
{
if(students[i].id == key)
{
pos = i;
break;
}
}
auto stop = high_resolution_clock::now(); auto linearTime =
duration_cast<nanoseconds>(stop - start);
if(pos != -1)
cout << "\nRecord Found Using Linear Search";
else
cout << "\nRecord Not Found";
// Bubble Sort by Name
for(int i = 0; i < n - 1; i++)
{
for(int j = 0; j < n - i - 1; j++)
{
if(strcmp(students[j].name, students[j + 1].name) > 0)
{
Student temp = students[j];
students[j] = students[j + 1];
students[j + 1] = temp;
}
}
}
```

```
cout << "\n\n===== Sorted By Name =====\n";
cout << "ID\tName\tCGPA\n";
for(int i = 0; i < n; i++)
{
cout << students[i].id << "\t"
<< students[i].name << "\t"
<< students[i].cgpa << endl; }
// Selection Sort by CGPA
for(int i = 0; i < n - 1; i++)
{
int min = i;
for(int j = i + 1; j < n; j++)
{
if(students[j].cgpa < students[min].cgpa)
min = j;
}
Student temp = students[i];
students[i] = students[min];
students[min] = temp;
}
cout << "\n\n===== Sorted By CGPA =====\n";
cout << "ID\tName\tCGPA\n";
for(int i = 0; i < n; i++)
{
cout << students[i].id << "\t"
<< students[i].name << "\t"
<< students[i].cgpa << endl;
}
```

```
// Sort by ID for Binary Search
for(int i = 0; i < n - 1; i++)
{
for(int j = 0; j < n - i - 1; j++)
{
if(students[j].id > students[j + 1].id)
{
Student temp = students[j];
students[j] = students[j + 1];
students[j + 1] = temp;
}
}
}
cout << "\nEnter Student ID to Search (Binary Search): ";
cin >> key;
start = high_resolution_clock::now();
int low = 0;
int high = n - 1;
pos = -1;
while(low <= high)
{
int mid = (low + high) / 2;
if(students[mid].id == key)
{
pos = mid; break;
}
else if(key < students[mid].id)
high = mid - 1;
```

```
else
low = mid + 1;
}
stop = high_resolution_clock::now();
auto binaryTime = duration_cast<nanoseconds>(stop - start);
if(pos != -1)
cout << "\nRecord Found Using Binary Search";
else
cout << "\nRecord Not Found";
// Performance Comparison
cout << "\n\n========== Performance Comparison ==========\n";
cout << "Linear Search Time : " << linearTime.count() << " ns\n";
cout << "Binary Search Time : " << binaryTime.count() << " ns\n";
free(students);
return 0;
}
```

Output:

Enter Number of Students: 5

Student 1

Enter ID: 105

Enter Name: Prachiti

Enter CGPA: 8.5

Student 2

Enter ID: 101

Enter Name: Amit

Enter CGPA: 9.2

Student 3

Enter ID: 103

Enter Name: Sakshi

Enter CGPA: 8.9

Student 4

Enter ID: 102

Enter Name: Neha

Enter CGPA: 7.8

Student 5

Enter ID: 104

Enter Name: Pranjal

Enter CGPA: 8.1

========== Student Records ==========

ID Name CGPA

105 Prachiti 8.5

101 Amit 9.2

103 Sakshi 8.9

102 Neha 7.8

104 Pranjal 8.1

Enter Student ID to Search (Linear Search): 103

Record Found Using Linear Search

===== Sorted By Name =====

ID Name CGPA

101 Amit 9.2

104 Pranjal 8.1

102 Neha 7.8

103 Sakshi 8.9

105 Prachiti 8.5

===== Sorted By CGPA =====

ID Name CGPA

102 Neha 7.8

104 Pranjal 8.1

105 Prachiti 8.5

103 Sakshi 8.9

101 Amit 9.2

Enter Student ID to Search (Binary Search): 103

Record Found Using Binary Search

========== **Performance Comparison** ==========

Linear Search Time : 1200 ns

Binary Search Time : 500 ns